OPERATING SYSTEMS – ASSIGNMENT 1

CO1: Evaluate and Justify the Operating-System Design of a Smart Hospital

Answer Report – Tasks 1 to 12

Student Name: JAGADISH.V Register Number:192521352

Course: CSA0417 – Operating Systems

Note: This report provides reasoned design answers and completed code based on the supplied assignment. Actual timings, strace counts and screenshots depend on the Linux computer used; record your own measured results before submission. Do not present illustrative values as measurements.

TASK 1 – Requirements of Each Computing Environment

A smart hospital has different workloads, so one general-purpose operating system cannot satisfy every requirement equally well. Patient safety and predictable response take priority for bedside monitors; availability and data integrity dominate the database; throughput is important for imaging; and energy efficiency matters for handheld tablets.

Environment	Ranked requirements (highest first)	Justification
Bedside monitors	1. Deadline ≤10 ms; 2. reliability; 3. sensor I/O; 4. security; 5. low resource use	A delayed alarm can affect patient safety. A hard real-time design must provide a bounded response time.
Nurses' tablets	1. security/privacy; 2. battery life; 3. reliable wireless communication; 4. usability; 5. responsiveness	Mobile use requires power management, encryption, authentication and reliable access to patient information.
Doctors' workstations	1. security; 2. interactive response; 3. compatibility; 4. reliability; 5. multitasking	Desktop systems support clinical applications, imaging viewers and multiple concurrent tasks.

Patient-record database	1. continuous availability; 2. integrity/recovery; 3. security; 4. throughput; 5. scalability	Records must remain accessible during maintenance and failures, with backups and controlled access.
MRI imaging server	1. high throughput; 2. parallel processing; 3. memory/I/O bandwidth; 4. reliability; 5. security	Large image workloads benefit from multiple processors, fast storage and DMA-capable I/O.
Partner-hospital link	1. confidentiality; 2. integrity; 3. availability; 4. interoperability; 5. scalability	Records cross organizational boundaries and need authenticated, encrypted and audited communication.

Conflicts and OS history: strict alarm deadlines conflict with best-effort time-sharing, while high throughput may compete with low latency for shared resources. Batch systems introduced queued processing and are useful for backups; time-sharing supports many interactive desktop users; personal-computer operating systems provide responsive graphical interfaces; mobile operating systems add aggressive power management and wireless security. Real-time monitor tasks should not be delayed by background work.

Alternative: A single general-purpose OS may be easier to administer and may reduce software variety, but ordinary scheduling does not guarantee a 10 ms worst-case alarm deadline. Its worst-case scheduling delay can be unbounded under overload or faulty drivers. A full OS also consumes more memory/storage on small monitors and tablets. Administration is simpler initially but tuning, patching and securing one OS across very different hardware is difficult. Separate fit-for-purpose OS choices reduce deadline risk and resource overhead.

TASK 2 – Type of Operating System for Each Environment

Environment	Recommended OS type	Rejected alternative / reason
Bedside monitors	Embedded hard real-time OS	General-purpose desktop OS may not guarantee the 10 ms deadline.

Nurses' tablets	Handheld/mobile OS	Desktop OS typically has greater power and resource demands.
Doctors' workstations	Desktop, time-sharing OS	A real-time OS is unnecessary for ordinary interactive clinical applications.
Database server	Clustered server OS with fail-over	One standalone server creates a single point of failure.
MRI imaging server	Multiprocessor OS, preferably SMP	A single processor limits parallel image processing.
Partner link	Distributed-system architecture	A single centralized machine does not support resilient cross-hospital sharing.

Mainframe systems are designed for large centralized workloads and can serve as a central transaction platform, but are not suitable for tiny bedside devices. Desktop systems suit individual workstations. Multiprocessor systems use multiple CPUs/cores. Distributed systems coordinate computers across a network. Clustered systems provide coordinated nodes and fail-over. Hard real-time systems must meet deadlines; soft real-time systems aim for timely service but occasional misses are tolerable. Handheld systems emphasize battery and mobility.

For MRI processing, Symmetric Multiprocessing (SMP) is preferred: processors share memory and the OS schedules work across them, simplifying load balancing. Asymmetric multiprocessing assigns specialized roles to processors and may suit a fixed embedded design, but is less flexible for a general imaging server. For the database, a cluster of smaller servers with replicated data and automatic fail-over is preferred. A single large multiprocessor server may offer strong throughput, but hardware or OS failure can stop the service. Cluster availability improves because another node can take over; cost and operational complexity are higher. Processor scaling is not perfectly linear because synchronization, storage and memory bandwidth become bottlenecks. Memory capacity is limited to the single machine in the standalone option, while a cluster can distribute workload and memory across nodes.

TASK 3 – Operating-System Structures

Structure	Reliability	Performance / updates

Monolithic	A faulty kernel component can affect the whole OS.	Fast internal calls; tightly coupled code makes changes riskier.
Layered	Clear boundaries help testing and fault isolation.	Layer crossings can add overhead; easier to reason about.
Microkernel	Small kernel limits trusted code; services run separately.	Message passing may add latency; services can often be restarted independently.
Modular	Modules can be loaded/unloaded; faults in kernel modules can still crash system.	Near-monolithic performance with flexible updates.
Hybrid	Combines selected monolithic and microkernel ideas.	Balances performance and modularity, but remains complex.

Recommendation: use a small microkernel or suitably certified modular real-time kernel for bedside monitors, because isolation and predictable behaviour are more important than minimizing every service-call overhead. Use a modular or hybrid server kernel for the database, where high I/O performance, mature drivers, monitoring and maintainability are important. Linux is generally monolithic with loadable kernel modules; Windows uses a hybrid architecture; Android uses the Linux kernel with the Android framework and HAL; FreeRTOS is a small real-time kernel, while QNX is a microkernel-based real-time OS.

Alternative: With a monolithic monitor kernel, a service request can be a direct system call and may be faster than microkernel message passing. A microkernel has a smaller privileged core, while a monolithic kernel includes more services and drivers in kernel space. A faulty monolithic driver can corrupt kernel memory or crash the whole monitor; in a microkernel design, an isolated driver/service failure may be contained and restarted. For patient safety, the isolation advantage outweighs modest message-passing overhead, provided the 10 ms deadline is verified.

TASK 4 – Operating-System Services

Environment	Services to include	Services to minimize / controls
Monitor	Sensor I/O, deterministic scheduling, alarms, watchdog, error detection, secure update, limited logging	No full desktop UI or unnecessary background services; signed software and restricted interfaces.
Tablet	Touch UI, app execution, file access, wireless communication, encryption, authentication, power management	Disable unused radios/services; enforce screen lock and managed updates.
Workstation	GUI, process management, file system, networking, printing, device drivers, security and audit	Restrict admin rights; use endpoint protection and automatic locking.
Database	Process, memory, storage, transaction I/O, networking, access control, audit, backup, recovery and accounting	Least privilege; encryption; replication; tested restore and fail-over.
Imaging server	Parallel execution, high-speed I/O, large-file system, networking, resource allocation and security	Prioritize imaging jobs without starving clinical services.

Patient-record isolation is enforced using authenticated user identities, role-based access control, process address-space protection, file permissions/ACLs, database permissions, encryption in transit and at rest, and audit logs. A user should access only the records required for their role. The bedside monitor can omit general-purpose file browsing, printing, rich desktop UI and unrelated services to reduce memory footprint and attack surface.

Alternative: A full desktop OS on tablets adds background services, larger storage and memory requirements, longer boot/start-up time and more update activity. These consume energy and can reduce battery endurance. A reduced handheld OS keeps only required clinical apps, communication, security and power-management services, improving battery life and simplifying administration.

TASK 5 – System Calls

System calls provide a controlled interface from user programs to kernel services. Main categories are: process control (fork, exec, wait, exit); file management (open, read, write, close); device management (ioctl and device I/O); information maintenance (getpid, time, system information); communication (pipe, socket, send/receive); and protection (chmod, access and identity-related calls).

A read() operation proceeds as follows: (1) the application calls the C library wrapper; (2) the wrapper places the system-call number and arguments in the required registers or memory; (3) a trap/syscall instruction transfers control to kernel mode; (4) the kernel validates the file descriptor and user buffer, obtains data from the file/device or cache, and copies data to the user buffer; (5) the kernel returns a result or error; (6) execution resumes in user mode. Parameters may be passed in registers, through a memory block whose address is passed in a register, or on the stack, depending on the architecture and ABI.

strace observes system calls and their return values, for example: strace -c ./copy. It is useful for checking which calls a program makes and their counts, but measurements vary by Linux version, file cache state and runtime environment.

Alternative: fopen(), fread() and fwrite() are C library functions that usually buffer data. open(), read() and write() are system calls. With a library buffer of about 4096 bytes, reading n bytes can require roughly n/4096 read system calls instead of roughly n/b calls for an application buffer of b bytes. The library also uses extra memory for its buffer. Library buffering is often efficient for sequential file access; direct system calls provide more explicit control.

TASK 6 – Cost of System Calls

A system call costs more than an ordinary function call because it crosses the user/kernel protection boundary, saves/restores execution state, validates arguments and may require scheduling or device interaction. Larger buffers reduce the number of calls and therefore reduce transition overhead.

The supplied test creates a 10,000-byte file. Expected read() counts are: 10,001 calls with a 1-byte buffer (10,000 data reads plus the final EOF read), 21 calls with a 512-byte buffer, and 4 calls with a 4096-byte buffer. A 4096-byte or larger practical buffer is a reasonable starting point for sequential backups, but the best size should be selected by measuring throughput, CPU time and memory use on the target server.

```c
/* copy_file.c – completed TODO 1 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

long copy_file(const char *src, const char *dst, size_t bufsize) {
    char buf[8192];
    if (bufsize == 0 || bufsize > sizeof buf) return -1;
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }

    long calls = 0;
    ssize_t n;
    while (1) {
        do { n = read(in, buf, bufsize); } while (n < 0 && errno == EINTR);
        calls++;
        if (n == 0) break;
        if (n < 0) { calls = -1; break; }

        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(out, buf + written, (size_t)(n - written));
            if (w < 0 && errno == EINTR) continue;
            if (w <= 0) { calls = -1; break; }
            written += w;
        }
        if (calls < 0) break;
    }
    close(in);
    close(out);
```

```
    return calls;
}
```

Compile the complete assignment program with: gcc -std=c11 -Wall -Wextra copy_file.c -o copy_file. Run: ./copy_file. For syscall evidence run: strace -c ./copy_file. The program should report the expected read counts listed above, subject to using the exact 10,000-byte test file and buffer sizes.

Alternative: mmap() maps a file into a process address space. It can reduce explicit read()/write() calls and copying between kernel and user buffers, but page faults still cause kernel work and very large mappings require address-space planning. A mapped copy may use a few setup calls rather than approximately n/b explicit reads, but it is not automatically faster for every workload. It uses virtual address space proportional to the mapped file range; physical pages are brought in as needed.

TASK 7 – Process Control System Calls

fork() creates a child process, usually using copy-on-write pages. exec() replaces the current process image with a new program. wait()/waitpid() lets a parent collect a child's termination status. exit() terminates a process and makes its status available to the parent. A shell typically parses a command, forks, execs the command in the child, waits in the parent for a foreground command, and reports the exit status.

A zombie is a terminated child whose exit status has not yet been collected by its parent; wait() or waitpid() reaps it. An orphan is a child whose parent terminates first; it is adopted by a system reaper process. fork() and exec() are separate so the child can set up file descriptors, redirection, environment and other process attributes before loading the requested program.

```
/* process_demo.c – completed TODO 1 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    printf("shell: running job 1\n");
    fflush(stdout);
    int status;
```

```
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        _exit(127);
    }
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) _exit(3);
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Compile and run: gcc -std=c11 -Wall -Wextra process_demo.c -o process_demo && ./process_demo. Expected output includes 42 from expr, followed by job 1 exit status 0 and job 2 exit status 3.

stdout is flushed before fork() so buffered text is not duplicated when both parent and child inherit the same output buffer. If the second fflush() is removed and the child calls exit(3), the child flushes its inherited copy of the buffered “shell: running job 2” text, and the parent later flushes its own copy; redirected output is fully buffered, so the line can appear twice. _exit() does not flush inherited stdio buffers.

Alternative: posix_spawn() can combine process creation and program loading with less application-visible setup and may be efficient for launching many simple commands. Threads share an address space and are cheaper to create, but they do not provide the same memory isolation as separate processes. For 1,000 commands, actual time and memory must be measured; fork() uses copy-on-write

rather than eagerly copying all memory, while threads share most process resources. Independent processes provide stronger fault and security isolation.

TASK 8 – CPU States and Protection

The CPU operates in user mode for applications and kernel mode for trusted OS code. A mode bit indicates the privilege level. Privileged instructions, such as disabling interrupts or changing memory-management settings, are permitted only in kernel mode. A user program requests privileged work through a system call/trap.

Hardware interrupts notify the CPU of external events such as sensor input or device completion. A trap is a synchronous transfer caused by an instruction or exception, including a system call. A timer interrupt lets the OS regain control from a running process and schedule another task. If arbitrary monitor software could disable interrupts, it could prevent timely sensor handling and delay alarms. Memory-management hardware (MMU) and page tables prevent one process from accessing another process's memory unless access is explicitly authorized.

Alternative: Polling checks the sensor repeatedly. With polling interval T, detection delay can approach T plus processing time; CPU cycles are also consumed while repeatedly checking. Interrupt-driven input can notify the CPU promptly without constant polling, although interrupt latency and scheduling still need a verified upper bound. For a strict 10 ms alarm requirement, choose a verified interrupt-driven real-time design and measure worst-case latency.

TASK 9 – Memory Hierarchy and I/O Systems

Level	Typical access time (approximate)	Typical role
CPU registers	Less than 1 ns	Current instructions and operands.
L1/L2/L3 cache	About 1–30 ns, depending on level	Frequently used instructions and data.
Main memory (RAM)	About 50–100 ns	Active programs, database cache and image buffers.
SSD	About 50–200 µs for random access (device-dependent)	Online records, application data and image storage.

Hard disk	About 5–15 ms for random access (device-dependent)	Bulk archive and backup.
Network storage	Sub-millisecond to many milliseconds, depending on network/load	Shared records, replication and off-site archive.

Keep immediate monitor readings and active working data in RAM/cache, then persist them to reliable local or central storage. Keep frequently accessed patient-record indexes and hot records in database memory/cache, with durable copies on SSD and replicated storage. Store MRI images on high-capacity fast storage with archival copies on lower-cost storage and backup. Do not rely on cache as the only copy of clinical data.

Programmed I/O makes the CPU repeatedly check device status and is suitable only for simple, low-volume devices. Interrupt-driven I/O lets the device notify the CPU when service is needed. DMA transfers blocks between a device and memory with limited CPU involvement, making it suitable for large MRI image transfers. Caching speeds record lookups by retaining frequently used data, but cached data must be invalidated or updated consistently.

Alternative: A 500 MB image contains 500 × 1024 × 1024 = 524,288,000 bytes. If interrupt-driven I/O generated one interrupt per 4-byte word, it would require about 131,072,000 interrupts; with 8-byte words, about 65,536,000. This would create substantial CPU overhead. DMA instead transfers blocks and generates far fewer completion interrupts. DMA needs appropriately sized buffers and careful synchronization; interrupt-driven I/O needs CPU service for each event and can require staging buffers. The one-interrupt-per-word scenario is deliberately inefficient.

TASK 10 – Measure and Compare

Use the following commands on Linux and enter the values actually observed on your machine. The assignment requires measurements, so the table below is a recording template rather than invented benchmark data.

gcc -std=c11 -Wall -Wextra copy_file.c -o copy_file\nstrace -c ./copy_file\ntime ./copy_file\n

Experiment	Measurement to record	Interpretation

Copy buffer 1 byte	read() calls: 10001 expected; elapsed/user/system time: _____	Many system calls increase transition overhead.
Copy buffer 512 bytes	read() calls: 21 expected; elapsed/user/system time: _____	Fewer calls should reduce overhead.
Copy buffer 4096 bytes	read() calls: 4 expected; elapsed/user/system time: _____	Usually a better balance for this small sequential test.
1,000 processes	Total creation/termination time: _____	Separate address spaces and process setup cost more.
1,000 threads	Total creation/termination time: _____	Threads share address space and are usually lighter.

The program's own counter counts read() calls made by copy_file(). strace -c may show additional reads made by the C runtime or dynamic loader, so compare the relevant syscall count carefully rather than assuming the total process count equals the function's counter. Run multiple trials, use the same input size, close unrelated applications where practical, and report the median. File caching can strongly affect timing.

Alternative: Threads usually require less memory per worker because code and heap are shared, while each thread still needs its own stack and thread-control data. Processes have separate virtual address spaces and additional process metadata, although fork() initially uses copy-on-write. Therefore thread creation is often faster and lighter, but processes offer better isolation. The measured difference depends on the OS, libraries, limits and workload.

TASK 11 – Recommend and Defend the Operating-System Plan

Hospital component	Recommended design and controls
Bedside monitors	Embedded hard real-time OS; small microkernel/RT kernel; bounded interrupt handling; watchdog; authenticated updates; minimal services.

Nurses' tablets	Managed handheld OS; encryption, device lock, secure wireless, remote update and power management.
Doctors' workstations	Desktop OS; role-based access, endpoint security, audit, automatic patching and controlled peripherals.
Patient database	Clustered server OS; replicated database, fail-over, backups, transaction logging, encryption and least privilege.
MRI imaging	SMP multiprocessor server; modular/hybrid OS; DMA, fast RAM/SSD and scheduled parallel workloads.
Hospital-to-hospital link	Distributed architecture; secure APIs, mutual authentication, encryption, audit and controlled replication.

This plan separates safety-critical real-time control from general-purpose computing. Its main limitations are multiple OS platforms to maintain, integration complexity, specialist support requirements and the need to validate real-time deadlines. The plan can scale from one ward to 500 monitors and then three hospitals by using device management, segmented networks, replicated databases, standardized interfaces and centralized monitoring. The distributed link must tolerate network outages and resolve synchronization conflicts safely.

Alternative: Moving everything into virtual machines on two large servers simplifies consolidation and may improve hardware utilization, snapshots and centralized management. However, virtualisation adds CPU and memory overhead, and contention or host failure can affect many services at once. A general-purpose hypervisor cannot automatically guarantee the bedside 10 ms deadline. Keep bedside monitors on dedicated embedded real-time hardware; virtualize suitable server and desktop workloads with resource reservations, redundant hosts and tested fail-over.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

Home monitoring introduces untrusted networks, variable latency, intermittent connectivity, device diversity and a need for remote device management. Cloud records introduce multi-tenant security, identity federation, data residency, service availability, backup and network-dependency concerns. The affected elements are the home-device OS, communication services, distributed database architecture, authentication, encryption, audit and update mechanisms.

B. Evaluation of the Original Design

The original embedded real-time monitor design remains appropriate for local alarm detection because it can continue operating even when the internet is unavailable. However, the hospital-only network and on-premises database assumptions are no longer sufficient. Internet latency cannot guarantee a 10 ms end-to-end alarm response, so urgent local alarms must be generated at the device or local gateway. Cloud records can improve sharing and scalability but depend on connectivity and provider availability.

C. Adaptation / Redesign

- RETAIN the embedded hard real-time OS for bedside monitors and use a similarly constrained embedded real-time OS for home devices where local safety alarms are required.
- ADD a secure home gateway or mobile connection that buffers readings during outages and forwards them when connectivity returns.
- MODIFY the tablet and workstation systems to use secure identity federation, multifactor authentication where appropriate, encrypted connections and centrally managed access policies.
- MODIFY the database tier into a distributed cloud/on-premises service with replicated data, backups, disaster recovery, audit trails and controlled synchronization among the three hospitals.
- USE secure APIs, mutual authentication, encryption in transit and at rest, least-privilege access, device certificates, signed updates and continuous security monitoring.

D. Evidence-Based Defence

The best decision is MODIFY rather than completely redesign. The real-time local control layer should be retained because cloud or internet delay is variable and cannot safely replace a guaranteed local alarm path. The data-sharing layer should be extended to a distributed cloud service because three hospitals and home devices need scalable, interoperable access. The trade-offs are higher integration and security-management complexity, cloud/network dependency and ongoing service costs. These risks are reduced through local buffering, offline operation, replication, tested recovery, encryption and strong access control.

Final conclusion: A hybrid architecture best fits the smart hospital: embedded real-time systems for patient monitoring, mobile and desktop systems for staff, clustered and distributed services for records, and multiprocessor systems for imaging. This architecture prioritizes patient safety, availability, privacy, performance and future growth.

Appendix – Suggested Linux Evidence and Submission Checklist

- Create repository: CSA04-<RegisterNumber>-Assignment1.
- Create separate folders Task01 through Task12; include source code, scripts, outputs and screenshots.
- Record compiler version (gcc --version), Linux distribution/version and exact compile commands.
- For Task 6, run the completed copy program and capture strace -c output.
- For Task 7, compile and run the process demonstration; capture the output and test the buffering explanation with redirected output.
- For Task 10, measure 1,000 process and thread creation trials using the same machine and report actual results.
- Commit progressively, at least one meaningful commit per task, and include the working GitHub URL in the report and repository README.

Important: The assignment asks for execution evidence and measurements. The explanations and code here are a written solution; compile and run the programs yourself, capture genuine terminal screenshots, fill in measured timings, and verify all outputs before submitting.